Modeling, Segmentation & Evaluation

Overview

This notebook focuses on building, comparing, and evaluating machine learning models for the Bank Marketing dataset. Building on the preprocessing and feature engineering steps from previous notebooks, we now develop predictive models to estimate the likelihood of a client subscribing to a term deposit.

In addition to a traditional global modeling approach, this notebook explores a segmentation-based strategy using clusters derived earlier. By comparing a single global model against cluster-specific models, we aim to understand whether segment-level specialization leads to improved predictive performance and better business outcomes.

Objectives

- Compare multiple candidate model architectures using the training and validation sets
- Select a primary modeling approach based on validation performance
- Train a global model using all available data and evaluate performance across clusters
- Train segment-specific models for each cluster and compare against the global model
- Perform hyperparameter tuning for both global and segment-specific models using validation data
- Train final models using selected hyperparameters
- Apply probability calibration to improve predicted probabilities
- Conduct cost-sensitive analysis using calibrated probabilities and different decision thresholds
- Evaluate final models on a held-out test set, including overall and cluster-level performance

Dataset Description

The modeling process uses the preprocessed dataset generated in the previous notebook, which includes engineered features such as categorical encodings, binary indicators, and interaction terms. Additionally, each observation is assigned to a cluster derived from unsupervised learning, enabling both global and segment-specific modeling approaches.

The data is split into three subsets:

- **Training set**: used to fit models
- **Validation set**: used for model selection, hyperparameter tuning, and calibration
- **Test set**: held out for final unbiased evaluation

The target variable represents whether a client subscribed to a term deposit.

Key Considerations

- **Model comparison**: Different model architectures are evaluated to identify the most suitable approach for the problem
- **Segmentation strategy**: Cluster-based segmentation allows us to investigate heterogeneity in client behavior and model performance
- **Fairness and consistency**: Performance is evaluated across clusters to detect potential disparities
- **Overfitting prevention**: Validation data is strictly used for model selection and hyperparameter tuning
- **Probability calibration**: Raw model probabilities may be poorly calibrated, which can negatively impact decision-making
- **Business alignment**: Model evaluation incorporates cost-sensitive analysis, considering trade-offs between false positives (e.g., unnecessary calls) and false negatives (missed opportunities)
- **Final evaluation integrity**: The test set remains untouched until the final evaluation step

Outcome

By the end of this notebook, we will have:

- Selected and trained a primary global model
- Developed segment-specific models tailored to different client clusters
- Tuned and calibrated all models for improved predictive reliability
- Compared global and segmented approaches from both predictive and business perspectives
- Identified optimal decision thresholds based on expected cost
- Produced a final evaluation on the test set, including overall performance, cluster-level insights, and cost-based metrics

In [1]:
```
import pandas as pd
import lightgbm as lgb
import optuna
import joblib
```

```python
from pathlib import Path
from sklearn.model_selection import ParameterGrid
from sklearn.isotonic import IsotonicRegression
from sklearn.metrics import average_precision_score

from utils.utils import load_dataset, evaluate_model, evaluate_from_probs

RANDOM_STATE = 705
```

```
/Users/tonantzinrealrojas/miniforge3/envs/nlp/lib/python3.13/site-packages/t
qdm/auto.py:21: TqdmWarning: IProgress not found. Please update jupyter and
ipywidgets. See https://ipywidgets.readthedocs.io/en/stable/user_install.htm
l
  from .autonotebook import tqdm as notebook_tqdm
```

In [ ]:
```python
# quarto preview 03_training.ipynb --to pdf
# quarto render 03_training.ipynb
# black 03_training.ipynb
```

1. Global architecture selection

In this section, we evaluate a set of candidate model architectures to determine which will be used in the subsequent comparison between a global model (trained on all observations) and segment-specific models (trained per cluster).

We begin by training and assessing five baseline architectures using their default configurations:

1. Logistic Regression
2. Random Forest
3. XGBoost
4. LightGBM
5. CatBoost

The goal is to identify a strong primary model that balances predictive performance and generalization before proceeding to more advanced analysis.

In [2]:
```python
import numpy as np
import pandas as pd

from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier

# from sklearn.metrics import (
#     average_precision_score,
#     f1_score,
#     precision_score,
#     recall_score,
# )

from xgboost import XGBClassifier
```

```
import lightgbm as lgb
from catboost import CatBoostClassifier
```

Because the outcome variable is highly imbalanced, we apply class weighting across all models to penalize misclassification of subscribers more heavily than non-subscribers. For tree-based models, this is implemented via the ratio of negative to positive samples, while linear and ensemble baselines use built-in balanced class weights. This approach preserves the original data distribution and produces probability estimates suitable for downstream calibration and cost-based decision analysis.

In [3]:
```
X_train_, y_train = load_dataset("02", "train")
X_validation_, y_validation = load_dataset("02", "validation")
```

```
train
X shape: (28934, 60)
y shape: (28934,)

X head:
```

	cat__job_admin.	cat__job_blue-collar	cat__job_entrepreneur	cat__job_housemaid	cat__
0	0.0	0.0	0.0	0.0	
1	0.0	1.0	0.0	0.0	
2	0.0	0.0	0.0	0.0	

3 rows × 60 columns

```
validation
X shape: (7234, 60)
y shape: (7234,)

X head:
```

	cat__job_admin.	cat__job_blue-collar	cat__job_entrepreneur	cat__job_housemaid	cat__
0	0.0	0.0	0.0	0.0	
1	0.0	1.0	0.0	0.0	
2	0.0	0.0	1.0	0.0	

3 rows × 60 columns

In [4]:
```
cluster_feat = "cluster"

X_train = X_train_.copy()
X_train = X_train.drop(columns=cluster_feat)
print("\nTrain set: ", X_train.shape)
display(X_train.head(3))

X_validation = X_validation_.copy()
X_validation = X_validation.drop(columns=cluster_feat).copy()
```

```
print("\nValidation set: ", X_validation.shape)
display(X_validation.head(3))
```

```
Train set:  (28934, 59)
```

	cat__job_admin.	cat__job_blue-collar	cat__job_entrepreneur	cat__job_housemaid	cat__
0	0.0	0.0	0.0	0.0	
1	0.0	1.0	0.0	0.0	
2	0.0	0.0	0.0	0.0	

3 rows × 59 columns

```
Validation set:  (7234, 59)
```

	cat__job_admin.	cat__job_blue-collar	cat__job_entrepreneur	cat__job_housemaid	cat__
0	0.0	0.0	0.0	0.0	
1	0.0	1.0	0.0	0.0	
2	0.0	0.0	1.0	0.0	

3 rows × 59 columns

In [5]:
```
n_pos = y_train.sum()
n_neg = len(y_train) - n_pos
scale_pos_weight = n_neg / n_pos
```

i. Logistic Regression

In [6]:
```
log_reg = LogisticRegression(
    penalty="l2",
    solver="liblinear",
    class_weight="balanced",
    max_iter=1000,
    random_state=RANDOM_STATE,
)

log_reg.fit(X_train, y_train)
log_reg_metrics = evaluate_model(log_reg, X_validation, y_validation)
```

ii. Random Forest

In [7]:
```
rf = RandomForestClassifier(
    n_estimators=100, class_weight="balanced", random_state=RANDOM_STATE, n_
)

rf.fit(X_train, y_train)
rf_metrics = evaluate_model(rf, X_validation, y_validation)
```

iii. XGBoost

In [8]:
```
xgb = XGBClassifier(
    objective="binary:logistic",
    eval_metric="aucpr",
    scale_pos_weight=scale_pos_weight,
    random_state=RANDOM_STATE,
)

xgb.fit(X_train, y_train)
xgb_metrics = evaluate_model(xgb, X_validation, y_validation)
```

iv. LightGBM

In [9]:
```
lgbm = lgb.LGBMClassifier(
    objective="binary", scale_pos_weight=scale_pos_weight, random_state=RAND
)

lgbm.fit(X_train, y_train)
lgbm_metrics = evaluate_model(lgbm, X_validation, y_validation)
```

```
[LightGBM] [Info] Number of positive: 3385, number of negative: 25549
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of te
sting was 0.001658 seconds.
You can set `force_row_wise=true` to remove the overhead.
And if memory is not enough, you can set `force_col_wise=true`.
[LightGBM] [Info] Total Bins 176
[LightGBM] [Info] Number of data points in the train set: 28934, number of u
sed features: 59
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.116990 -> initscore=-2.021
244
[LightGBM] [Info] Start training from score -2.021244
```

v. CatBoost

In [10]:
```
cat = CatBoostClassifier(
    loss_function="Logloss",
    eval_metric="PRAUC",
    class_weights=[1, scale_pos_weight],
    verbose=0,
    random_state=RANDOM_STATE,
)

cat.fit(X_train, y_train)
cat_metrics = evaluate_model(cat, X_validation, y_validation)
```

Results comparison

In [13]:
```
results = pd.DataFrame.from_dict(
    {
        "Logistic Regression": log_reg_metrics,
```

```
        "Random Forest": rf_metrics,
        "XGBoost": xgb_metrics,
        "LightGBM": lgbm_metrics,
        "CatBoost": cat_metrics,
    },
    orient="index",
)

results.sort_values(by="AUC_PR", ascending=False)
```

Out[13]:

	AUC_PR	F1	Precision	Recall
LightGBM	0.445832	0.459201	0.362826	0.625296
CatBoost	0.442484	0.459831	0.368159	0.612293
Logistic Regression	0.411748	0.395126	0.287325	0.632388
XGBoost	0.407729	0.434281	0.347795	0.578014
Random Forest	0.313136	0.275808	0.413712	0.206856

On the validation set, LightGBM and CatBoost exhibit very similar performance, with nearly identical F1 scores and only a marginal difference in AUC-PR (0.446 for LightGBM versus 0.442 for CatBoost). Both models clearly outperform the other architectures and appear well suited to handling the strong class imbalance present in the data.

LightGBM was selected as the primary model architecture because it achieves the highest AUC-PR overall, indicating slightly better ranking of likely subscribers, while maintaining strong recall and competitive precision. Given the minimal performance gap, this choice is pragmatic rather than definitive; CatBoost remains a strong alternative and could be expected to perform similarly in downstream analysis. **LightGBM** is therefore used for the remainder of the project to provide a consistent baseline for both global and segment-specific modeling.

2. Global model

Following the architecture selection, we now develop a fully optimized Global Model. This model serves as the primary benchmark for the project, trained on the complete dataset to capture broad customer patterns. To manage the significant class imbalance (~11.7% success rate), we implement an analytical class-weighting strategy (scale_pos_weight) that penalizes errors on the minority class more heavily, ensuring the model prioritizes potential subscribers over the majority of non-subscribers.

Once we decided to work with LightGBM, we'll train the global model which will the observations from all clusters.

In [2]:

```
X_train_, y_train = load_dataset("02", "train")
X_validation_, y_validation = load_dataset("02", "validation")
```

```
# X_test_, y_test_ = load_dataset(nb="02", split="test")
```

```
train
X shape: (28934, 60)
y shape: (28934,)

X head:
```

	cat__job_admin.	cat__job_blue-collar	cat__job_entrepreneur	cat__job_housemaid	cat__
0	0.0	0.0	0.0	0.0	
1	0.0	1.0	0.0	0.0	
2	0.0	0.0	0.0	0.0	

3 rows × 60 columns

```
validation
X shape: (7234, 60)
y shape: (7234,)

X head:
```

	cat__job_admin.	cat__job_blue-collar	cat__job_entrepreneur	cat__job_housemaid	cat__
0	0.0	0.0	0.0	0.0	
1	0.0	1.0	0.0	0.0	
2	0.0	0.0	1.0	0.0	

3 rows × 60 columns

In [3]:
```
cluster_feat = "cluster"

X_train = X_train_.copy()
X_train = X_train.drop(columns=cluster_feat)
print("\nTrain set: ", X_train.shape)
display(X_train.head(3))

X_validation = X_validation_.copy()
X_validation = X_validation.drop(columns=cluster_feat).copy()
print("\nValidation set: ", X_validation.shape)
display(X_validation.head(3))

# X_test = X_test_.copy()
# X_test = X_test.drop(columns=cluster_feat).copy()
# print("\nTest set: ", X_test.shape)
# display(X_test.head(3))
```

```
Train set:  (28934, 59)
```

	cat__job_admin.	cat__job_blue-collar	cat__job_entrepreneur	cat__job_housemaid	cat__
0	0.0	0.0	0.0	0.0	
1	0.0	1.0	0.0	0.0	
2	0.0	0.0	0.0	0.0	

3 rows × 59 columns

```
Validation set:  (7234, 59)
```

	cat__job_admin.	cat__job_blue-collar	cat__job_entrepreneur	cat__job_housemaid	cat__
0	0.0	0.0	0.0	0.0	
1	0.0	1.0	0.0	0.0	
2	0.0	0.0	1.0	0.0	

3 rows × 59 columns

In [4]:
```
n_pos = y_train.sum()
n_neg = len(y_train) - n_pos
scale_pos_weight = n_neg / n_pos

print(f"Scale pos weight: {scale_pos_weight:.2f}")
```

```
Scale pos weight: 7.55
```

1. Hyperparameter Optimization via Optuna

We employ Bayesian Optimization using the Optuna framework to fine-tune the LightGBM hyperparameters. Unlike a standard grid search, this approach efficiently explores the parameter space to maximize the Area Under the Precision-Recall Curve (AUC-PR). This specific metric is chosen to ensure high model reliability and ranking performance within the minority positive class, which is essential for effective telemarketing targeting.

In [5]:
```
def objective(trial):
    params = {
        "objective": "binary",
        "metric": "aucpr",
        "random_state": RANDOM_STATE,
        "scale_pos_weight": scale_pos_weight,
        "verbosity": -1,
        # hyperparameters to tune
        "num_leaves": trial.suggest_int("num_leaves", 16, 128),
        "max_depth": trial.suggest_int("max_depth", -1, 12),
        "learning_rate": trial.suggest_float("learning_rate", 0.01, 0.2, log
        "n_estimators": trial.suggest_int("n_estimators", 200, 600),
        "min_child_samples": trial.suggest_int("min_child_samples", 20, 200)
```

```
    }

    model = lgb.LGBMClassifier(**params)
    model.fit(X_train, y_train)

    y_val_prob = model.predict_proba(X_validation)[:, 1]
    aucpr = average_precision_score(y_validation, y_val_prob)

    return aucpr
```

In [6]:

```
study = optuna.create_study(direction="maximize")
study.optimize(objective, n_trials=30, show_progress_bar=True)

print("Best AUC-PR:", study.best_value)
print("Best parameters:", study.best_params)

best_params = study.best_params
```

```
[I 2026-04-10 23:04:27,987] A new study created in memory with name: no-name
-e246ef53-0664-4780-888f-c749538ed4bc
Best trial: 0. Best value: 0.44695:   3%|▌          | 1/30 [00:01<00:30,  1.0
4s/it]
[I 2026-04-10 23:04:29,046] Trial 0 finished with value: 0.44695013721562915
and parameters: {'num_leaves': 84, 'max_depth': 5, 'learning_rate': 0.074318
37596167686, 'n_estimators': 331, 'min_child_samples': 159}. Best is trial 0
with value: 0.44695013721562915.
Best trial: 0. Best value: 0.44695:   7%|▌          | 2/30 [00:04<01:08,  2.4
6s/it]
[I 2026-04-10 23:04:32,490] Trial 1 finished with value: 0.4388096066105165
and parameters: {'num_leaves': 83, 'max_depth': 0, 'learning_rate': 0.020686
55937226802, 'n_estimators': 234, 'min_child_samples': 163}. Best is trial 0
with value: 0.44695013721562915.
Best trial: 2. Best value: 0.44942:  10%|▌          | 3/30 [00:05<00:50,  1.8
5s/it]
[I 2026-04-10 23:04:33,631] Trial 2 finished with value: 0.44942040819902246
and parameters: {'num_leaves': 27, 'max_depth': 6, 'learning_rate': 0.077391
8301859272, 'n_estimators': 260, 'min_child_samples': 29}. Best is trial 2 w
ith value: 0.44942040819902246.
Best trial: 2. Best value: 0.44942:  13%|▌          | 4/30 [00:13<01:46,  4.1
0s/it]
[I 2026-04-10 23:04:41,179] Trial 3 finished with value: 0.44543984423293637
and parameters: {'num_leaves': 84, 'max_depth': 0, 'learning_rate': 0.020616
43916609482, 'n_estimators': 514, 'min_child_samples': 149}. Best is trial 2
with value: 0.44942040819902246.
Best trial: 2. Best value: 0.44942:  17%|▌          | 5/30 [00:15<01:23,  3.3
5s/it]
[I 2026-04-10 23:04:43,206] Trial 4 finished with value: 0.43388064695894607
and parameters: {'num_leaves': 101, 'max_depth': 8, 'learning_rate': 0.08378
316239668203, 'n_estimators': 285, 'min_child_samples': 100}. Best is trial
2 with value: 0.44942040819902246.
Best trial: 2. Best value: 0.44942:  20%|▌          | 6/30 [00:15<00:54,  2.2
9s/it]
```

```
[I 2026-04-10 23:04:43,429] Trial 5 finished with value: 0.40551218420962865
and parameters: {'num_leaves': 114, 'max_depth': 1, 'learning_rate': 0.03889
8611034226614, 'n_estimators': 432, 'min_child_samples': 76}. Best is trial
2 with value: 0.44942040819902246.
Best trial: 2. Best value: 0.44942:  23%|██        | 7/30 [00:18<00:57,  2.5
1s/it]
[I 2026-04-10 23:04:46,396] Trial 6 finished with value: 0.43360102360513375
and parameters: {'num_leaves': 63, 'max_depth': 10, 'learning_rate': 0.06156
0002107802346, 'n_estimators': 330, 'min_child_samples': 108}. Best is trial
2 with value: 0.44942040819902246.
Best trial: 2. Best value: 0.44942:  27%|██        | 8/30 [00:21<00:57,  2.6
1s/it]
[I 2026-04-10 23:04:49,215] Trial 7 finished with value: 0.44322486423680874
and parameters: {'num_leaves': 100, 'max_depth': 7, 'learning_rate': 0.05438
3707087731076, 'n_estimators': 495, 'min_child_samples': 90}. Best is trial
2 with value: 0.44942040819902246.
Best trial: 2. Best value: 0.44942:  30%|└└└       | 9/30 [00:22<00:46,  2.2
1s/it]
[I 2026-04-10 23:04:50,559] Trial 8 finished with value: 0.44714013050246026
and parameters: {'num_leaves': 37, 'max_depth': 6, 'learning_rate': 0.056921
200987851125, 'n_estimators': 340, 'min_child_samples': 177}. Best is trial
2 with value: 0.44942040819902246.
Best trial: 2. Best value: 0.44942:  33%|└└└       | 10/30 [00:32<01:34,  4.
73s/it]
[I 2026-04-10 23:05:00,914] Trial 9 finished with value: 0.408006756899824 a
nd parameters: {'num_leaves': 120, 'max_depth': 0, 'learning_rate': 0.073985
19220157099, 'n_estimators': 519, 'min_child_samples': 161}. Best is trial 2
with value: 0.44942040819902246.
Best trial: 2. Best value: 0.44942:  37%|└└└       | 11/30 [00:33<01:06,  3.
49s/it]
[I 2026-04-10 23:05:01,603] Trial 10 finished with value: 0.4401951217556178
7 and parameters: {'num_leaves': 16, 'max_depth': 12, 'learning_rate': 0.148
82574957389713, 'n_estimators': 212, 'min_child_samples': 31}. Best is trial
2 with value: 0.44942040819902246.
Best trial: 2. Best value: 0.44942:  40%|└└└└      | 12/30 [00:34<00:51,  2.
85s/it]
[I 2026-04-10 23:05:02,989] Trial 11 finished with value: 0.4253470682412467
and parameters: {'num_leaves': 32, 'max_depth': 5, 'learning_rate': 0.147502
67765943173, 'n_estimators': 389, 'min_child_samples': 29}. Best is trial 2
with value: 0.44942040819902246.
Best trial: 2. Best value: 0.44942:  43%|└└└└      | 13/30 [00:35<00:35,  2.
10s/it]
[I 2026-04-10 23:05:03,368] Trial 12 finished with value: 0.4364124810413397
3 and parameters: {'num_leaves': 47, 'max_depth': 3, 'learning_rate': 0.0352
5384573018336, 'n_estimators': 283, 'min_child_samples': 62}. Best is trial
2 with value: 0.44942040819902246.
Best trial: 2. Best value: 0.44942:  47%|└└└└      | 14/30 [00:38<00:39,  2.
49s/it]
[I 2026-04-10 23:05:06,738] Trial 13 finished with value: 0.4217712902174249
and parameters: {'num_leaves': 40, 'max_depth': 8, 'learning_rate': 0.108661
86048023993, 'n_estimators': 595, 'min_child_samples': 198}. Best is trial 2
with value: 0.44942040819902246.
```

```
Best trial: 2. Best value: 0.44942:  50%|[redacted]     | 15/30 [00:39<00:28,  1.
88s/it]
[I 2026-04-10 23:05:07,222] Trial 14 finished with value: 0.4200843245135355
and parameters: {'num_leaves': 18, 'max_depth': 3, 'learning_rate': 0.011878
475484227758, 'n_estimators': 378, 'min_child_samples': 199}. Best is trial
2 with value: 0.44942040819902246.
Best trial: 2. Best value: 0.44942:  53%|[redacted]     | 16/30 [00:40<00:23,  1.
66s/it]
[I 2026-04-10 23:05:08,374] Trial 15 finished with value: 0.4447443237526201
3 and parameters: {'num_leaves': 60, 'max_depth': 6, 'learning_rate': 0.0279
69329841806664, 'n_estimators': 282, 'min_child_samples': 126}. Best is tria
l 2 with value: 0.44942040819902246.
Best trial: 16. Best value: 0.45746:  57%|[redacted]     | 17/30 [00:41<00:17,
1.38s/it]
[I 2026-04-10 23:05:09,097] Trial 16 finished with value: 0.4574604157509598
6 and parameters: {'num_leaves': 31, 'max_depth': 4, 'learning_rate': 0.0990
8788645704905, 'n_estimators': 348, 'min_child_samples': 55}. Best is trial
16 with value: 0.45746041575095986.
Best trial: 16. Best value: 0.45746:  60%|[redacted]     | 18/30 [00:41<00:12,
1.07s/it]
[I 2026-04-10 23:05:09,435] Trial 17 finished with value: 0.4558562645119725
3 and parameters: {'num_leaves': 54, 'max_depth': 3, 'learning_rate': 0.1075
0220926221801, 'n_estimators': 253, 'min_child_samples': 54}. Best is trial
16 with value: 0.45746041575095986.
Best trial: 16. Best value: 0.45746:  67%|[redacted]     | 20/30 [00:42<00:06,
1.44it/s]
[I 2026-04-10 23:05:09,997] Trial 18 finished with value: 0.4532695228741327
and parameters: {'num_leaves': 52, 'max_depth': 3, 'learning_rate': 0.189387
30267869036, 'n_estimators': 438, 'min_child_samples': 54}. Best is trial 16
with value: 0.45746041575095986.
[I 2026-04-10 23:05:10,172] Trial 19 finished with value: 0.4410533882724813
and parameters: {'num_leaves': 72, 'max_depth': 2, 'learning_rate': 0.113882
59726749773, 'n_estimators': 200, 'min_child_samples': 55}. Best is trial 16
with value: 0.45746041575095986.
Best trial: 16. Best value: 0.45746:  70%|[redacted]     | 21/30 [00:42<00:06,
1.44it/s]
[I 2026-04-10 23:05:10,870] Trial 20 finished with value: 0.4489777646317958
4 and parameters: {'num_leaves': 51, 'max_depth': 4, 'learning_rate': 0.1066
934909106978, 'n_estimators': 359, 'min_child_samples': 129}. Best is trial
16 with value: 0.45746041575095986.
Best trial: 16. Best value: 0.45746:  73%|[redacted]     | 22/30 [00:43<00:04,
1.69it/s]
[I 2026-04-10 23:05:11,227] Trial 21 finished with value: 0.4518024021210253
and parameters: {'num_leaves': 53, 'max_depth': 2, 'learning_rate': 0.182031
46672800333, 'n_estimators': 441, 'min_child_samples': 51}. Best is trial 16
with value: 0.45746041575095986.
Best trial: 16. Best value: 0.45746:  77%|[redacted]     | 23/30 [00:44<00:04,
1.46it/s]
[I 2026-04-10 23:05:12,132] Trial 22 finished with value: 0.4370886479793330
6 and parameters: {'num_leaves': 68, 'max_depth': 4, 'learning_rate': 0.1953
4764926655376, 'n_estimators': 434, 'min_child_samples': 77}. Best is trial
16 with value: 0.45746041575095986.
```

```
Best trial: 16. Best value: 0.45746:  80%|▌▌▌▌▌▌▌▌  | 24/30 [00:44<00:03,
1.69it/s]
[I 2026-04-10 23:05:12,504] Trial 23 finished with value: 0.4495939796419402
and parameters: {'num_leaves': 44, 'max_depth': 2, 'learning_rate': 0.140243
02830898192, 'n_estimators': 460, 'min_child_samples': 44}. Best is trial 16
with value: 0.45746041575095986.
Best trial: 16. Best value: 0.45746:  83%|▌▌▌▌▌▌▌▌  | 25/30 [00:46<00:05,
1.04s/it]
[I 2026-04-10 23:05:14,587] Trial 24 finished with value: 0.4352195865080950
6 and parameters: {'num_leaves': 28, 'max_depth': -1, 'learning_rate': 0.097
29814048195962, 'n_estimators': 413, 'min_child_samples': 73}. Best is trial
16 with value: 0.45746041575095986.
Best trial: 16. Best value: 0.45746:  87%|▌▌▌▌▌▌▌▌  | 26/30 [00:47<00:03,
1.09it/s]
[I 2026-04-10 23:05:15,230] Trial 25 finished with value: 0.4530741071241400
4 and parameters: {'num_leaves': 57, 'max_depth': 4, 'learning_rate': 0.1354
7726592535755, 'n_estimators': 303, 'min_child_samples': 44}. Best is trial
16 with value: 0.45746041575095986.
Best trial: 16. Best value: 0.45746:  93%|▌▌▌▌▌▌▌▌▌ | 28/30 [00:48<00:01,
1.58it/s]
[I 2026-04-10 23:05:15,870] Trial 26 finished with value: 0.4478586712199646
and parameters: {'num_leaves': 33, 'max_depth': 3, 'learning_rate': 0.190199
8437319316, 'n_estimators': 477, 'min_child_samples': 21}. Best is trial 16
with value: 0.45746041575095986.
[I 2026-04-10 23:05:16,035] Trial 27 finished with value: 0.4103414592636812
6 and parameters: {'num_leaves': 74, 'max_depth': 1, 'learning_rate': 0.1277
7240113068722, 'n_estimators': 249, 'min_child_samples': 92}. Best is trial
16 with value: 0.45746041575095986.
Best trial: 16. Best value: 0.45746:  97%|▌▌▌▌▌▌▌▌▌▌| 29/30 [00:49<00:00,
1.04it/s]
[I 2026-04-10 23:05:17,761] Trial 28 finished with value: 0.4420159306992519
4 and parameters: {'num_leaves': 44, 'max_depth': 5, 'learning_rate': 0.0921
6093320614571, 'n_estimators': 557, 'min_child_samples': 67}. Best is trial
16 with value: 0.45746041575095986.
Best trial: 16. Best value: 0.45746: 100%|▌▌▌▌▌▌▌▌▌▌| 30/30 [00:50<00:00,
1.68s/it]
[I 2026-04-10 23:05:18,429] Trial 29 finished with value: 0.4478119721002908
3 and parameters: {'num_leaves': 23, 'max_depth': 4, 'learning_rate': 0.1618
314366611486, 'n_estimators': 327, 'min_child_samples': 84}. Best is trial 1
6 with value: 0.45746041575095986.
Best AUC-PR: 0.45746041575095986
Best parameters: {'num_leaves': 31, 'max_depth': 4, 'learning_rate': 0.09908
788645704905, 'n_estimators': 348, 'min_child_samples': 55}
```

We train the final global base model with the best parameters

In [7]:
```
global_model = lgb.LGBMClassifier(
    objective="binary",
    scale_pos_weight=scale_pos_weight,
    random_state=RANDOM_STATE,
    **best_params
)
```

```
global_model.fit(X_train, y_train)
```

Out[7]:

```
▾ LGBMClassifier
▶ Parameters
```

2. Probability Calibration

Machine learning models, particularly those using class-weighting, often produce "scores" rather than true probabilities. We apply Isotonic Regression to calibrate these outputs, aligning the model's predicted likelihoods with the actual observed subscription rates. Well-calibrated probabilities are a prerequisite for our upcoming Business Cost Framework, where we will calculate expected ROI based on specific call costs and subscription values.

In [8]:

```
# raw probabilities on validation
val_probs_raw = global_model.predict_proba(X_validation)[:, 1]

# isotonic regression calibration
calibrator = IsotonicRegression(out_of_bounds="clip")
calibrator.fit(val_probs_raw, y_validation)

# calibrated probabilities
val_probs_calibrated = calibrator.transform(val_probs_raw)
```

In [9]:

```
global_validation_metrics = evaluate_from_probs(y_validation, val_probs_cali

print("Global model (validation, calibrated):")
print(global_validation_metrics)
```

```
Global model (validation, calibrated):
{'AUC_PR': 0.4457539936060022, 'F1': 0.2846441947565543, 'Precision': 0.6846
846846846847, 'Recall': 0.17966903073286053}
```

3. Baseline Performance across Customer Segments

To evaluate the "one-size-fits-all" approach, we project the Global Model's performance onto our four distinct customer clusters. This diagnostic step identifies where the global patterns are robust and where they fail to capture localized nuances.

In [10]:

```
cluster_results = {}

for c in sorted(X_validation_[cluster_feat].unique()):
    idx = X_validation_[cluster_feat] == c

    cluster_results[f"cluster_{c}"] = evaluate_from_probs(
        y_validation.loc[idx], val_probs_calibrated[idx]
    )
```

```
cluster_results_df = pd.DataFrame(cluster_results).T
cluster_results_df
```

Out[10]:

	AUC_PR	F1	Precision	Recall
cluster_0	0.563676	0.387597	0.757576	0.260417
cluster_1	0.295621	0.171717	0.629630	0.099415
cluster_2	0.380164	0.243478	0.608696	0.152174
cluster_3	0.495403	0.298429	0.686747	0.190635

In [11]:

```
model_dir = Path("models")

model_name = "global"

# save base model
joblib.dump(global_model, model_dir / f"{model_name}_model.joblib")

# save calibrator
joblib.dump(calibrator, model_dir / f"{model_name}_calibrator.joblib")

# save metadata
metadata = {
    "best_params": best_params,
    "scale_pos_weight": scale_pos_weight,
    "features": list(X_train.columns),
    "random_state": RANDOM_STATE,
}

joblib.dump(metadata, model_dir / f"{model_name}_metadata.joblib")
```

Out[11]: ['models/global_metadata.joblib']

3. Cluster 0 model

In [2]:

```
cluster_id = 0
X_train, y_train = load_dataset(nb="02", split="train", cluster_id=cluster_i
X_validation, y_validation = load_dataset(
    nb="02", split="validation", cluster_id=cluster_id
)
# X_test, y_test = load_dataset(nb="02", split="test", cluster_id=cluster_ic
```

```
train | cluster 0
X shape: (5043, 59)
y shape: (5043,)

X head:
```

	cat__job_admin.	cat__job_blue-collar	cat__job_entrepreneur	cat__job_housemaid	cat_
2	0.0	0.0	0.0	0.0	
6	0.0	0.0	0.0	0.0	
10	0.0	0.0	0.0	0.0	

3 rows × 59 columns

```
validation | cluster 0
X shape: (1271, 59)
y shape: (1271,)

X head:
```

	cat__job_admin.	cat__job_blue-collar	cat__job_entrepreneur	cat__job_housemaid	cat__
1	0.0	1.0	0.0	0.0	
2	0.0	0.0	1.0	0.0	
3	0.0	0.0	0.0	0.0	

3 rows × 59 columns

In [3]:
```
# 2. Calculate the specific scale_pos_weight for Cluster 0
# Segment-specific weighting is essential as subscription rates vary by clus
n_pos = y_train.sum()
n_neg = len(y_train) - n_pos
scale_pos_weight = n_neg / n_pos

print(f"Cluster {cluster_id} - Training Shape: {X_train.shape}")
print(f"Cluster {cluster_id} - Scale Pos Weight: {scale_pos_weight:.2f}")
```

```
Cluster 0 - Training Shape: (5043, 59)
Cluster 0 - Scale Pos Weight: 6.14
```

1. Hyperparameter Optimization via Optuna

In [4]:
```
def objective(trial):
    params = {
        "objective": "binary",
        "metric": "aucpr",
        "random_state": RANDOM_STATE,
        "scale_pos_weight": scale_pos_weight,
        "verbosity": -1,
        # hyperparameters to tune
        "num_leaves": trial.suggest_int("num_leaves", 16, 128),
        "max_depth": trial.suggest_int("max_depth", -1, 12),
        "learning_rate": trial.suggest_float("learning_rate", 0.01, 0.2, log
        "n_estimators": trial.suggest_int("n_estimators", 200, 600),
        "min_child_samples": trial.suggest_int("min_child_samples", 20, 200)
    }
```

```
    model = lgb.LGBMClassifier(**params)
    model.fit(X_train, y_train)

    y_val_prob = model.predict_proba(X_validation)[:, 1]
    aucpr = average_precision_score(y_validation, y_val_prob)

    return aucpr
```

In [5]:

```
study = optuna.create_study(direction="maximize")
study.optimize(objective, n_trials=30, show_progress_bar=True)

print("Best AUC-PR:", study.best_value)
print("Best parameters:", study.best_params)

best_params = study.best_params
```

```
[I 2026-04-11 07:28:57,585] A new study created in memory with name: no-name
-8e38ff80-4890-4747-84d9-82b21c84db51
Best trial: 0. Best value: 0.535751:   3%|▌          | 1/30 [00:00<00:12,  2.
32it/s]
[I 2026-04-11 07:28:58,026] Trial 0 finished with value: 0.5357513535198656
and parameters: {'num_leaves': 105, 'max_depth': 3, 'learning_rate': 0.02021
0533861844435, 'n_estimators': 286, 'min_child_samples': 91}. Best is trial
0 with value: 0.5357513535198656.
Best trial: 0. Best value: 0.535751:   7%|▌          | 2/30 [00:02<00:41,  1.
50s/it]
[I 2026-04-11 07:29:00,268] Trial 1 finished with value: 0.5240057342779578
and parameters: {'num_leaves': 53, 'max_depth': 9, 'learning_rate': 0.048503
408146279015, 'n_estimators': 397, 'min_child_samples': 96}. Best is trial 0
with value: 0.5357513535198656.
Best trial: 2. Best value: 0.550214:  10%|▌          | 3/30 [00:04<00:47,  1.
77s/it]
[I 2026-04-11 07:29:02,356] Trial 2 finished with value: 0.5502144217246995
and parameters: {'num_leaves': 16, 'max_depth': 11, 'learning_rate': 0.04378
644661164682, 'n_estimators': 551, 'min_child_samples': 141}. Best is trial
2 with value: 0.5502144217246995.
Best trial: 2. Best value: 0.550214:  13%|▌          | 4/30 [00:06<00:50,  1.
92s/it]
[I 2026-04-11 07:29:04,523] Trial 3 finished with value: 0.5309289366508524
and parameters: {'num_leaves': 36, 'max_depth': 11, 'learning_rate': 0.05168
8605929131576, 'n_estimators': 401, 'min_child_samples': 141}. Best is trial
2 with value: 0.5502144217246995.
Best trial: 2. Best value: 0.550214:  17%|▌          | 5/30 [00:07<00:39,  1.
58s/it]
[I 2026-04-11 07:29:05,497] Trial 4 finished with value: 0.5429202368067308
and parameters: {'num_leaves': 33, 'max_depth': 5, 'learning_rate': 0.017458
093513231396, 'n_estimators': 373, 'min_child_samples': 174}. Best is trial
2 with value: 0.5502144217246995.
Best trial: 5. Best value: 0.553069:  20%|▌          | 6/30 [00:10<00:43,  1.
83s/it]
```

```
[I 2026-04-11 07:29:07,795] Trial 5 finished with value: 0.5530694182369915
and parameters: {'num_leaves': 16, 'max_depth': 10, 'learning_rate': 0.01182
9034817050152, 'n_estimators': 599, 'min_child_samples': 164}. Best is trial
5 with value: 0.5530694182369915.
Best trial: 5. Best value: 0.553069:  23%|██        | 7/30 [00:12<00:44,  1.
94s/it]
[I 2026-04-11 07:29:09,977] Trial 6 finished with value: 0.5434827197772043
and parameters: {'num_leaves': 121, 'max_depth': 10, 'learning_rate': 0.0114
37850616199409, 'n_estimators': 265, 'min_child_samples': 68}. Best is trial
5 with value: 0.5530694182369915.
Best trial: 5. Best value: 0.553069:  27%|██        | 8/30 [00:18<01:13,  3.
32s/it]
[I 2026-04-11 07:29:16,245] Trial 7 finished with value: 0.48368229317803424
and parameters: {'num_leaves': 91, 'max_depth': 10, 'learning_rate': 0.04906
094092289272, 'n_estimators': 540, 'min_child_samples': 25}. Best is trial 5
with value: 0.5530694182369915.
Best trial: 5. Best value: 0.553069:  30%|██        | 9/30 [00:19<00:50,  2.
42s/it]
[I 2026-04-11 07:29:16,692] Trial 8 finished with value: 0.5452119916070283
and parameters: {'num_leaves': 122, 'max_depth': 2, 'learning_rate': 0.01957
466578728525, 'n_estimators': 386, 'min_child_samples': 146}. Best is trial
5 with value: 0.5530694182369915.
Best trial: 5. Best value: 0.553069:  33%|██        | 10/30 [00:19<00:37,
1.86s/it]
[I 2026-04-11 07:29:17,291] Trial 9 finished with value: 0.5301448904438327
and parameters: {'num_leaves': 105, 'max_depth': 3, 'learning_rate': 0.01357
9401354164165, 'n_estimators': 358, 'min_child_samples': 61}. Best is trial
5 with value: 0.5530694182369915.
Best trial: 5. Best value: 0.553069:  37%|└──        | 11/30 [00:21<00:33,
1.76s/it]
[I 2026-04-11 07:29:18,830] Trial 10 finished with value: 0.4933227388218776
6 and parameters: {'num_leaves': 71, 'max_depth': 7, 'learning_rate': 0.1830
1367214197278, 'n_estimators': 488, 'min_child_samples': 198}. Best is trial
5 with value: 0.5530694182369915.
Best trial: 5. Best value: 0.553069:  40%|└──        | 12/30 [00:23<00:35,
1.98s/it]
[I 2026-04-11 07:29:21,303] Trial 11 finished with value: 0.5185977656611258
and parameters: {'num_leaves': 17, 'max_depth': 12, 'learning_rate': 0.09835
736200211118, 'n_estimators': 599, 'min_child_samples': 138}. Best is trial
5 with value: 0.5530694182369915.
Best trial: 12. Best value: 0.553247:  43%|└──        | 13/30 [00:25<00:35,
2.06s/it]
[I 2026-04-11 07:29:23,562] Trial 12 finished with value: 0.5532474225944112
and parameters: {'num_leaves': 16, 'max_depth': -1, 'learning_rate': 0.03061
2566719765602, 'n_estimators': 595, 'min_child_samples': 171}. Best is trial
12 with value: 0.5532474225944112.
Best trial: 12. Best value: 0.553247:  47%|└──        | 14/30 [00:28<00:33,
2.11s/it]
[I 2026-04-11 07:29:25,777] Trial 13 finished with value: 0.537917983102569
and parameters: {'num_leaves': 49, 'max_depth': -1, 'learning_rate': 0.03066
050959854648, 'n_estimators': 473, 'min_child_samples': 178}. Best is trial
12 with value: 0.5532474225944112.
```

```
Best trial: 12. Best value: 0.553247:  50%|[redacted]     | 15/30 [00:31<00:35,
2.35s/it]
[I 2026-04-11 07:29:28,681] Trial 14 finished with value: 0.5379109015539879
and parameters: {'num_leaves': 33, 'max_depth': -1, 'learning_rate': 0.01000
9858386846588, 'n_estimators': 595, 'min_child_samples': 170}. Best is trial
12 with value: 0.5532474225944112.
Best trial: 12. Best value: 0.553247:  53%|[redacted]     | 16/30 [00:32<00:29,
2.10s/it]
[I 2026-04-11 07:29:30,204] Trial 15 finished with value: 0.5295382777319643
and parameters: {'num_leaves': 63, 'max_depth': 7, 'learning_rate': 0.028978
3138761311, 'n_estimators': 476, 'min_child_samples': 196}. Best is trial 12
with value: 0.5532474225944112.
Best trial: 12. Best value: 0.553247:  57%|[redacted]     | 17/30 [00:32<00:20,
1.56s/it]
[I 2026-04-11 07:29:30,498] Trial 16 finished with value: 0.5244615521911272
and parameters: {'num_leaves': 26, 'max_depth': 1, 'learning_rate': 0.090511
69028663533, 'n_estimators': 524, 'min_child_samples': 117}. Best is trial 1
2 with value: 0.5532474225944112.
Best trial: 12. Best value: 0.553247:  60%|[redacted]     | 18/30 [00:33<00:15,
1.26s/it]
[I 2026-04-11 07:29:31,062] Trial 17 finished with value: 0.546893882157164
and parameters: {'num_leaves': 47, 'max_depth': 5, 'learning_rate': 0.024904
464009587016, 'n_estimators': 214, 'min_child_samples': 161}. Best is trial
12 with value: 0.5532474225944112.
Best trial: 12. Best value: 0.553247:  63%|[redacted]     | 19/30 [00:35<00:16,
1.47s/it]
[I 2026-04-11 07:29:33,031] Trial 18 finished with value: 0.524778466112854
and parameters: {'num_leaves': 86, 'max_depth': 8, 'learning_rate': 0.084994
90806844091, 'n_estimators': 440, 'min_child_samples': 117}. Best is trial 1
2 with value: 0.5532474225944112.
Best trial: 12. Best value: 0.553247:  67%|[redacted]     | 20/30 [00:38<00:18,
1.87s/it]
[I 2026-04-11 07:29:35,833] Trial 19 finished with value: 0.5374756903800708
and parameters: {'num_leaves': 23, 'max_depth': 0, 'learning_rate': 0.014641
238645922969, 'n_estimators': 554, 'min_child_samples': 159}. Best is trial
12 with value: 0.5532474225944112.
Best trial: 12. Best value: 0.553247:  70%|[redacted]     | 21/30 [00:39<00:14,
1.60s/it]
[I 2026-04-11 07:29:36,805] Trial 20 finished with value: 0.5377783782334162
and parameters: {'num_leaves': 60, 'max_depth': 4, 'learning_rate': 0.033181
71118592264, 'n_estimators': 509, 'min_child_samples': 188}. Best is trial 1
2 with value: 0.5532474225944112.
Best trial: 12. Best value: 0.553247:  73%|[redacted]     | 22/30 [00:41<00:14,
1.79s/it]
[I 2026-04-11 07:29:39,048] Trial 21 finished with value: 0.5370270435514309
and parameters: {'num_leaves': 17, 'max_depth': 12, 'learning_rate': 0.06541
123959544205, 'n_estimators': 557, 'min_child_samples': 131}. Best is trial
12 with value: 0.5532474225944112.
Best trial: 12. Best value: 0.553247:  77%|[redacted]     | 23/30 [00:43<00:12,
1.85s/it]
```

```
[I 2026-04-11 07:29:41,040] Trial 22 finished with value: 0.5361292694270815
and parameters: {'num_leaves': 42, 'max_depth': 7, 'learning_rate': 0.037205
72285470496, 'n_estimators': 575, 'min_child_samples': 155}. Best is trial 1
2 with value: 0.5532474225944112.
Best trial: 12. Best value: 0.553247:  80%|▌▌▌▌▌▌▌▌  | 24/30 [00:46<00:12,
2.11s/it]
[I 2026-04-11 07:29:43,764] Trial 23 finished with value: 0.5024281591507643
and parameters: {'num_leaves': 24, 'max_depth': 9, 'learning_rate': 0.130197
07923847532, 'n_estimators': 598, 'min_child_samples': 124}. Best is trial 1
2 with value: 0.5532474225944112.
Best trial: 12. Best value: 0.553247:  83%|▌▌▌▌▌▌▌▌  | 25/30 [00:48<00:10,
2.11s/it]
[I 2026-04-11 07:29:45,865] Trial 24 finished with value: 0.5456090509676413
and parameters: {'num_leaves': 16, 'max_depth': 11, 'learning_rate': 0.02447
6809406153186, 'n_estimators': 563, 'min_child_samples': 177}. Best is trial
12 with value: 0.5532474225944112.
Best trial: 12. Best value: 0.553247:  87%|▌▌▌▌▌▌▌▌  | 26/30 [00:49<00:07,
1.89s/it]
[I 2026-04-11 07:29:47,239] Trial 25 finished with value: 0.5393990122284769
and parameters: {'num_leaves': 38, 'max_depth': 6, 'learning_rate': 0.065453
51386496262, 'n_estimators': 446, 'min_child_samples': 147}. Best is trial 1
2 with value: 0.5532474225944112.
Best trial: 12. Best value: 0.553247:  90%|▌▌▌▌▌▌▌▌▌ | 27/30 [00:52<00:06,
2.16s/it]
[I 2026-04-11 07:29:50,015] Trial 26 finished with value: 0.5324374741517625
and parameters: {'num_leaves': 29, 'max_depth': 9, 'learning_rate': 0.038125
8898372249, 'n_estimators': 517, 'min_child_samples': 102}. Best is trial 12
with value: 0.5532474225944112.
Best trial: 12. Best value: 0.553247:  93%|▌▌▌▌▌▌▌▌▌ | 28/30 [00:55<00:04,
2.32s/it]
[I 2026-04-11 07:29:52,732] Trial 27 finished with value: 0.5298370633139018
and parameters: {'num_leaves': 24, 'max_depth': 11, 'learning_rate': 0.05891
838528043975, 'n_estimators': 572, 'min_child_samples': 165}. Best is trial
12 with value: 0.5532474225944112.
Best trial: 12. Best value: 0.553247:  97%|▌▌▌▌▌▌▌▌▌▌| 29/30 [00:57<00:02,
2.22s/it]
[I 2026-04-11 07:29:54,720] Trial 28 finished with value: 0.5453560467966511
and parameters: {'num_leaves': 43, 'max_depth': 8, 'learning_rate': 0.024939
19954169482, 'n_estimators': 530, 'min_child_samples': 184}. Best is trial 1
2 with value: 0.5532474225944112.
Best trial: 12. Best value: 0.553247: 100%|▌▌▌▌▌▌▌▌▌▌| 30/30 [00:57<00:00,
1.92s/it]
[I 2026-04-11 07:29:55,191] Trial 29 finished with value: 0.5429181306706922
and parameters: {'num_leaves': 78, 'max_depth': 3, 'learning_rate': 0.041027
45775483835, 'n_estimators': 313, 'min_child_samples': 75}. Best is trial 12
with value: 0.5532474225944112.
Best AUC-PR: 0.5532474225944112
Best parameters: {'num_leaves': 16, 'max_depth': -1, 'learning_rate': 0.0306
12566719765602, 'n_estimators': 595, 'min_child_samples': 171}
```

In [6]:

```
model = lgb.LGBMClassifier(
    objective="binary",
    scale_pos_weight=scale_pos_weight,
    random_state=RANDOM_STATE,
```

```
    **best_params
)

model.fit(X_train, y_train)
```

Out[6]:

```
▾ LGBMClassifier
▶ Parameters
```

2. Probability Calibration

In [7]:

```
# raw probabilities on validation
val_probs_raw = model.predict_proba(X_validation)[:, 1]

# isotonic regression calibration
calibrator = IsotonicRegression(out_of_bounds="clip")
calibrator.fit(val_probs_raw, y_validation)

# calibrated probabilities
val_probs_calibrated = calibrator.transform(val_probs_raw)
```

In [9]:

```
c0_validation_metrics = evaluate_from_probs(y_validation, val_probs_calibrat

print(f"Cluster {cluster_id} model (validation, calibrated):")
print(c0_validation_metrics)
```

```
Cluster 0 model (validation, calibrated):
{'AUC_PR': 0.553079139992169, 'F1': 0.4198473282442748, 'Precision': 0.78571
42857142857, 'Recall': 0.2864583333333333}
```

We save the model

In [11]:

```
model_dir = Path("models")

model_name = f"c{cluster_id}"

# save base model
joblib.dump(model, model_dir / f"{model_name}_model.joblib")

# save calibrator
joblib.dump(calibrator, model_dir / f"{model_name}_calibrator.joblib")

# save metadata
metadata = {
    "best_params": best_params,
    "scale_pos_weight": scale_pos_weight,
    "features": list(X_train.columns),
    "random_state": RANDOM_STATE,
}

joblib.dump(metadata, model_dir / f"{model_name}_metadata.joblib")
```

Out[11]:

```
['models/c0_metadata.joblib']
```

4. Cluster 1 model

In [ ]:
```
cluster_id = 1
X_train, y_train = load_dataset(nb="02", split="train", cluster_id=cluster_i
X_validation, y_validation = load_dataset(
    nb="02", split="validation", cluster_id=cluster_id
)
X_test, y_test = load_dataset(nb="02", split="test", cluster_id=cluster_id)
```

5. Cluster 2 model

In [ ]:
```
cluster_id = 2
X_train, y_train = load_dataset(nb="02", split="train", cluster_id=cluster_i
X_validation, y_validation = load_dataset(
    nb="02", split="validation", cluster_id=cluster_id
)
X_test, y_test = load_dataset(nb="02", split="test", cluster_id=cluster_id)
```

6. Cluster 3 model

In [ ]:
```
cluster_id = 3
X_train, y_train = load_dataset(nb="02", split="train", cluster_id=cluster_i
X_validation, y_validation = load_dataset(
    nb="02", split="validation", cluster_id=cluster_id
)
X_test, y_test = load_dataset(nb="02", split="test", cluster_id=cluster_id)
```